JavaGuide面试突击（2025最新版）

设计模式篇

JavaGuide

公众号：JavaGuide

网站：javaguide.cn

前言

由于很多读者都有突击面试的需求，所以我在几年前就弄了 **JavaGuide 面试突击版本**（JavaGuide 内容精简版，只保留重点），并持续完善跟进。对于喜欢纸质阅读的朋友来说，也可以打印出来，整体阅读体验非常高！

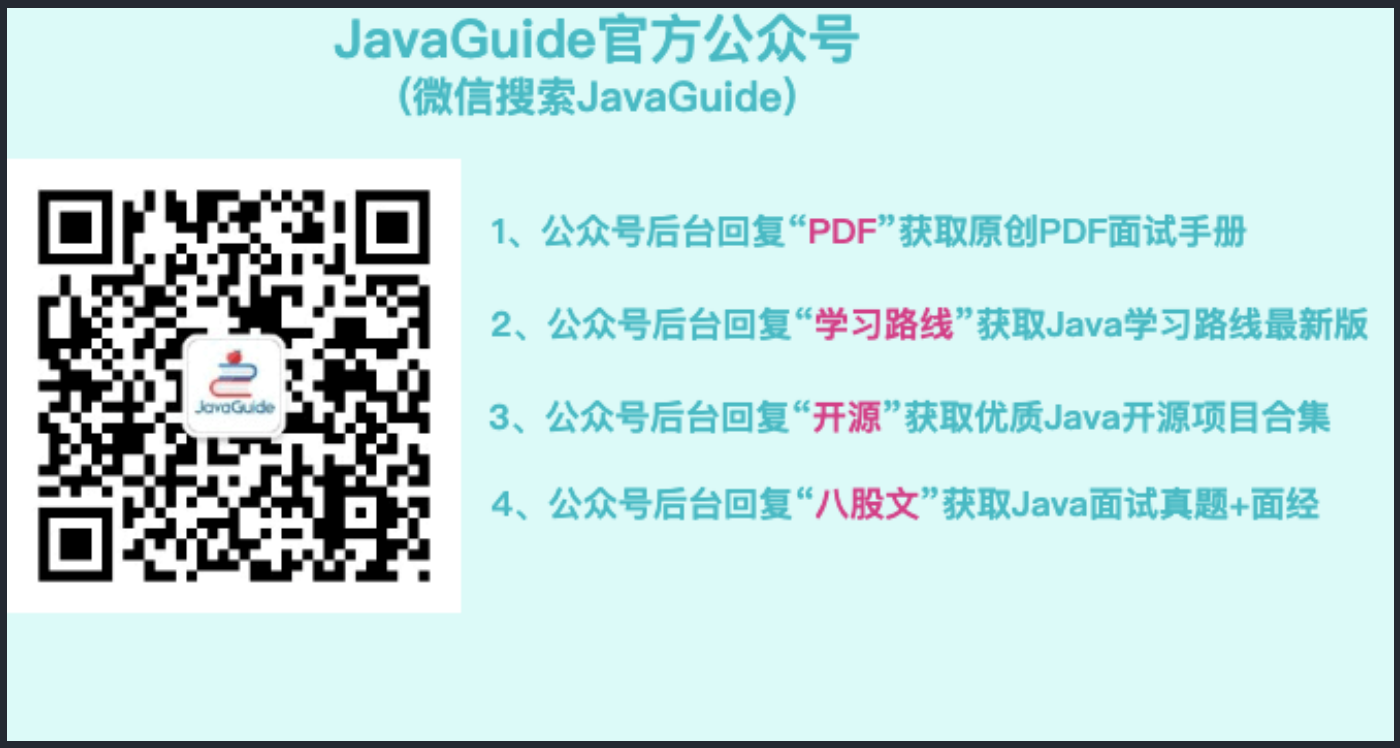
除了只保留最常问的面试题之外，我还进一步对重点中的重点进行了⭐标注。并且，有亮色（白天）和暗色（夜间）两个主题选择，需要打印出来的朋友记得选择亮色版本。

对于时间比较充裕的朋友，我个人还是更推荐 JavaGuide 网站系统学习，内容更全面，更深入。

JavaGuide 已经持续维护 6 年多了，累计提交了接近 **6000** commit ，共有 **570+** 多位贡献者共同参与维护和完善。用心做原创优质内容，如果觉得有帮助的话，欢迎点赞分享！传送门：GitHub | Gitee。

对于需要更进一步面试辅导服务的读者，欢迎加入 **JavaGuide 官方知识星球**(技术专栏/一对一提问/简历修改/求职指南/面试打卡)，绝对物超所值！

面试突击最新版本可以在我的公众号回复“**PDF**”获取（JavaGuide 官方知识星球会提前同步最新版，针对球友的一个小福利）。



软件设计原则有哪些?

在软件开发中，我们遵循一系列面向对象设计原则来提升代码的可维护性、可复用性和可扩展性，从而构建出更加健壮和灵活的系统。这些原则中最核心、最著名的就是 **SOLID** 原则，同时还有几个非常重要的补充原则。

SOLID 原则：

1. **单一职责原则 (Single Responsibility Principle, SRP)**
    - **核心思想**：一个类或模块应该有且只有一个引起它变化的原因。
    - **通俗来说**：一个类只做一件相关的事情。如果一个类承担了多个职责，那么当其中一个职责发生变化时，可能会影响到其他职责的实现，这会增加代码的复杂性和耦合度。
2. **开闭原则 (Open-Closed Principle, OCP)**
    - **核心思想**：软件实体（类、模块、函数等）应该对扩展开放，对修改关闭。
    - **通俗来说**：当需要增加新功能时，我们应该通过增加新的代码（例如新的子类或实现类）来实现，而不是去修改已有的、工作良好的代码。这是面向对象设计中最重要的一条原则，是实现系统可复用性和可维护性的基石。
3. **里氏替换原则 (Liskov Substitution Principle, LSP)**
    - **核心思想**：所有引用基类的地方，都必须能够透明地使用其子类的对象。
    - **通俗来说**：子类应当可以完全替换父类，并且程序的行为不会产生任何错误或异常。这意味着子类在继承父类时，不应该改变父类预期的行为。它是实现开闭原则的重要方式之一。
4. **接口隔离原则 (Interface Segregation Principle, ISP)**
    - **核心思想**：客户端不应该被强迫依赖它不需要的接口。一个类对另一个类的依赖应该建立在最小的接口之上。
    - **通俗来说**：接口的设计要“专”，不要设计大而全的“胖接口”，而应该根据功能设计多个细粒度的接口。这样可以避免一个类为了实现接口中的某一个功能，而不得不实现所有其他它并不需要的功能。
5. **依赖倒置原则 (Dependency Inversion Principle, DIP)**
    - **核心思想**：高层模块不应该依赖于低层模块，两者都应该依赖于抽象；抽象不应该依赖于细节，细节应该依赖于抽象。
    - **通俗来说**：我们要面向接口编程，而不是面向实现编程。代码的依赖关系应该通过抽象（如接口或抽象类）来建立，而不是具体的实现类。这样可以大大降低模块间的耦合度，方便系统的升级和扩展。

其他重要原则：

1. **合成/聚合复用原则 (Composite/Aggregate Reuse Principle, CARP)**
    - **核心思想**：尽量使用对象组合（合成/聚合）的方式，而不是继承来达到代码复用的目的。
    - **通俗来说**：“has-a” 关系通常比 “is-a” 关系更灵活。继承是一种强耦合关系（白盒复用），而组合是一种弱耦合关系（黑盒复用），后者更容易维护和扩展。
2. **迪米特法则 (Law of Demeter, LoD) / 最少知识原则 (Least Knowledge Principle, LKP)**
    - **核心思想**：一个软件实体应当尽可能少地与其他实体发生相互作用。
    - **通俗来说**：一个对象应该对其他对象有尽可能少的了解，即“只和你的直接朋友交谈”。这有助于降低类之间的耦合度，让每个模块更加独立。

这些原则并非孤立存在，而是相辅相成，共同指导我们写出“高内聚、低耦合”的优质代码。

什么是设计模式?

设计模式（Design Pattern）可以理解为一套**在特定场景下，针对软件设计中常见问题的、可复用的解决方案**。

它不是一个可以直接转换成代码的最终设计，而更像一个**蓝图或模板**，描述了如何组织类和对象来解决某个特定的设计难题。

我们可以从以下几个角度来理解它：

1. **它是经验的总结**：设计模式是无数软件工程师在长期实践中，经过反复试验和验证，总结出的一套行之有效的“最佳实践”。
2. **它是沟通的语言**：设计模式为开发者提供了一套通用的词汇。当你说“这里我用了一个工厂模式”，团队里的其他成员能立刻理解你的设计意图，这极大地提高了沟通效率。
3. **它解决了什么问题**：它的核心目标是提升代码的**可重用性**、**可读性**、**健壮性**和**可维护性**。通过遵循设计模式，我们可以写出结构更清晰、耦合度更低、更容易扩展和修改的代码，避免重复“造轮子”和踩一些常见的“坑”。

总而言之，设计模式是软件工程化的重要基石，它帮助我们将代码设计从“个人技艺”提升到“工程规范”的层面。

设计模式的分类了解吗?

根据设计模式的目的和关注点的不同，它们通常被分为三大类，这是由《设计模式：可复用面向对象软件的基础》这本书（常被称为 "GoF 四人帮" 的著作）提出的经典分类：

1. **创建型模式 (Creational Patterns)**
    - **关注点**：**对象的创建过程**。这类模式将对象的创建和使用解耦，使得程序在创建对象时更具灵活性。它们隐藏了对象创建的复杂逻辑，使得我们无需直接使用 new 关键字。
    - **核心思想**：提供一种机制，使得客户端代码不必关心它所需要的具体是哪个类的实例，也不必关心这些实例是如何被创建和组织的。
    - **常见模式**：
        - 单例模式 (Singleton)
        - 工厂方法模式 (Factory Method)
        - 抽象工厂模式 (Abstract Factory)
        - 建造者模式 (Builder)
        - 原型模式 (Prototype)
2. **结构型模式 (Structural Patterns)**
    - **关注点**：**类和对象的组合**。这类模式研究如何将类和对象组合在一起，形成更大、更复杂的结构，同时保持结构的灵活性和效率。
    - **核心思想**：通过继承、组合等方式，在不改变原有类的情况下，为其增加新的功能或适配不同的接口。
    - **常见模式**：
        - 适配器模式 (Adapter)
        - 桥接模式 (Bridge)
        - 组合模式 (Composite)
        - 装饰器模式 (Decorator)
        - 外观模式 (Facade)
        - 享元模式 (Flyweight)
        - 代理模式 (Proxy)
3. **行为型模式 (Behavioral Patterns)**
    - **关注点**：**对象之间的通信和职责分配**。这类模式专门处理对象之间的交互和协作，以及算法和责任的分配。
    - **核心思想**：通过定义对象间的通信模式来降低它们之间的耦合度，使得系统中的对象可以独立地变化。
    - **常见模式**：
        - 责任链模式 (Chain of Responsibility)

- 命令模式 (Command)
- 迭代器模式 (Iterator)
- 中介者模式 (Mediator)
- 备忘录模式 (Memento)
- 观察者模式 (Observer)
- 状态模式 (State)
- 策略模式 (Strategy)
- 模板方法模式 (Template Method)
- 访问者模式 (Visitor)
- 解释器模式 (Interpreter)

工厂模式

请谈一谈你对工厂模式的理解

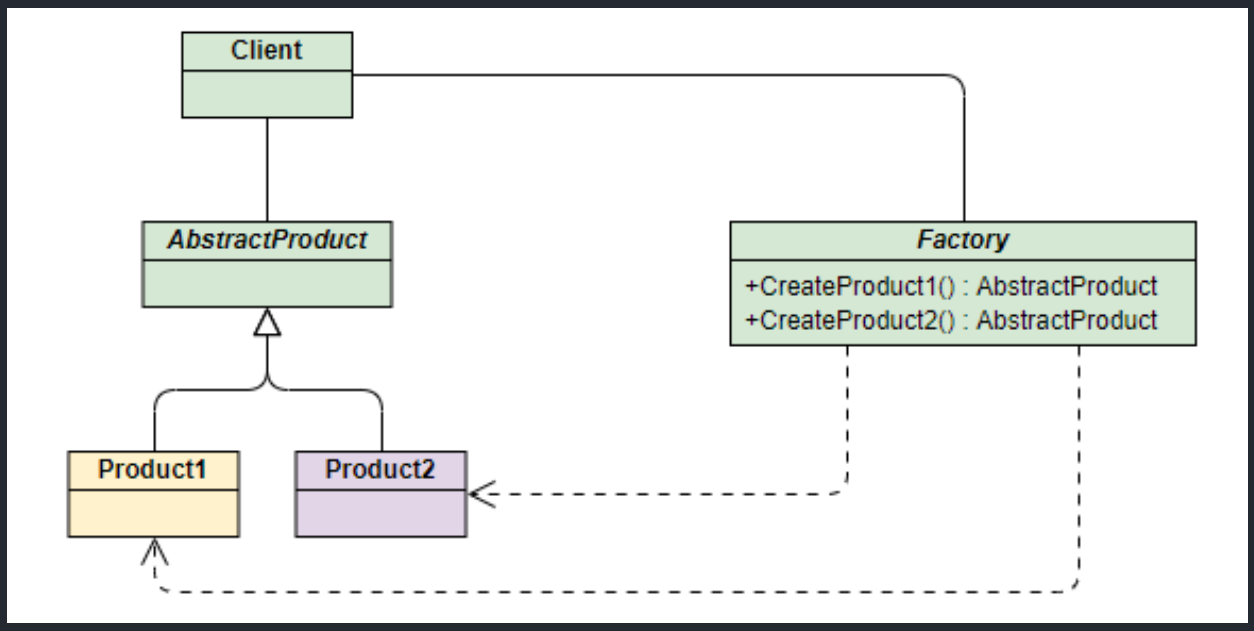
工厂模式是创建型设计模式中最常用的一族，它主要解决的是**对象的创建问题**，将对象的创建和使用过程进行解耦。根据其复杂度和应用场景的不同，通常分为三种：

1. 简单工厂模式
2. 工厂方法模式
3. 抽象工厂模式

说一说简单工厂模式

简单工厂模式指由一个**单一的工厂对象**来创建实例，客户端不需要关注创建逻辑，只需提供传入工厂的参数。

UML 类图如下：



- **角色组成**：
  - **工厂 (Factory)**：负责实现创建所有实例的内部逻辑。
  - **抽象产品 (AbstractProduct)**：所有被创建对象的父类或接口。
  - **具体产品 (ConcreteProduct)**：工厂类创建的目标实例。
- **解决的问题**：将客户端代码与具体产品的实现类解耦。客户端只需“消费”产品，而无需关心产品是如何被创建的。
- **优缺点**：
  - **优点**：结构简单，将创建逻辑集中管理，实现了职责分离。
  - **缺点**：**违反了开闭原则**。当需要增加新产品时，必须修改工厂类内部的判断逻辑（例如 if-else 或 switch 语句），这使得工厂类越来越臃肿，难以维护。
- **实例**：
  - Java 的 `Calendar.getInstance()` 方法，内部会根据时区、地区等参数创建具体的日历对象。
  - Spring 的 `BeanFactory` 在概念上就是一个强大的简单工厂，你通过一个 bean 的名字（ID）就能获取到对应的实例，而无需关心这个 bean 是如何被创建和配置的。

简单工厂代码实现：

```
// 抽象产品：电子设备
interface Device {
    void operate(); // 操作设备
}
```

```java
// 具体产品：手机
class Phone implements Device {
    @Override
    public void operate() {
        System.out.println("手机操作：开机 -> 显示主界面 -> 关机\n");
    }
}

// 具体产品：电脑
class Computer implements Device {
    @Override
    public void operate() {
        System.out.println("电脑操作：开机 -> 加载系统 -> 关机\n");
    }
}

// 简单工厂：设备工厂
class DeviceFactory {
    // 根据类型创建设备
    public static Device createDevice(String type) {
        if (type.equalsIgnoreCase("PHONE")) {
            return new Phone();
        } else if (type.equalsIgnoreCase("COMPUTER")) {
            return new Computer();
        }
        return null;
    }
}

public class SimpleFactoryDemo {
    public static void main(String[] args) {
        // 通过工厂创建设备
        Device phone = DeviceFactory.createDevice("PHONE");
        Device computer = DeviceFactory.createDevice("COMPUTER");

        // 使用设备
        phone.operate();
        computer.operate();
    }
```

```
    }
```

输出结果：

```
手机操作：开机 -> 显示主界面 -> 关机

电脑操作：开机 -> 加载系统 -> 关机
```

静态工厂和简单工厂的区别

静态工厂（Static Factory） 是一种基于**类的静态方法**创建对象的方式，属于《Effective Java》中推荐的创建对象的技巧（严格来说不算标准设计模式，但常被归类为创建型模式）。

上面的简单工厂模式代码用静态工厂改写如下：

```java
// 抽象产品：电子设备
interface Device {
    void operate(); // 操作设备
}

// 具体产品：手机（包含静态工厂方法）
class Phone implements Device {
    // 私有构造函数，强制通过静态工厂创建
    private Phone() {}

    // 静态工厂方法：创建手机实例
    public static Phone createPhone() {
        return new Phone();
    }

    @Override
    public void operate() {
        System.out.println("手机操作：开机 -> 显示主界面 -> 关机\n");
    }
}
```

```java
// 具体产品：电脑（包含静态工厂方法）
class Computer implements Device {
    // 私有构造函数，强制通过静态工厂创建
    private Computer() {}

    // 静态工厂方法：创建电脑实例
    public static Computer createComputer() {
        return new Computer();
    }

    @Override
    public void operate() {
        System.out.println("电脑操作：开机 -> 加载系统 -> 关机\n");
    }
}

// 客户端使用
public class StaticFactoryDemo {
    public static void main(String[] args) {
        // 通过静态工厂方法直接创建设备
        Device phone = Phone.createPhone();
        Device computer = Computer.createComputer();

        // 使用设备
        phone.operate();
        computer.operate();
    }
}
```

静态工厂和简单工厂对比如下：

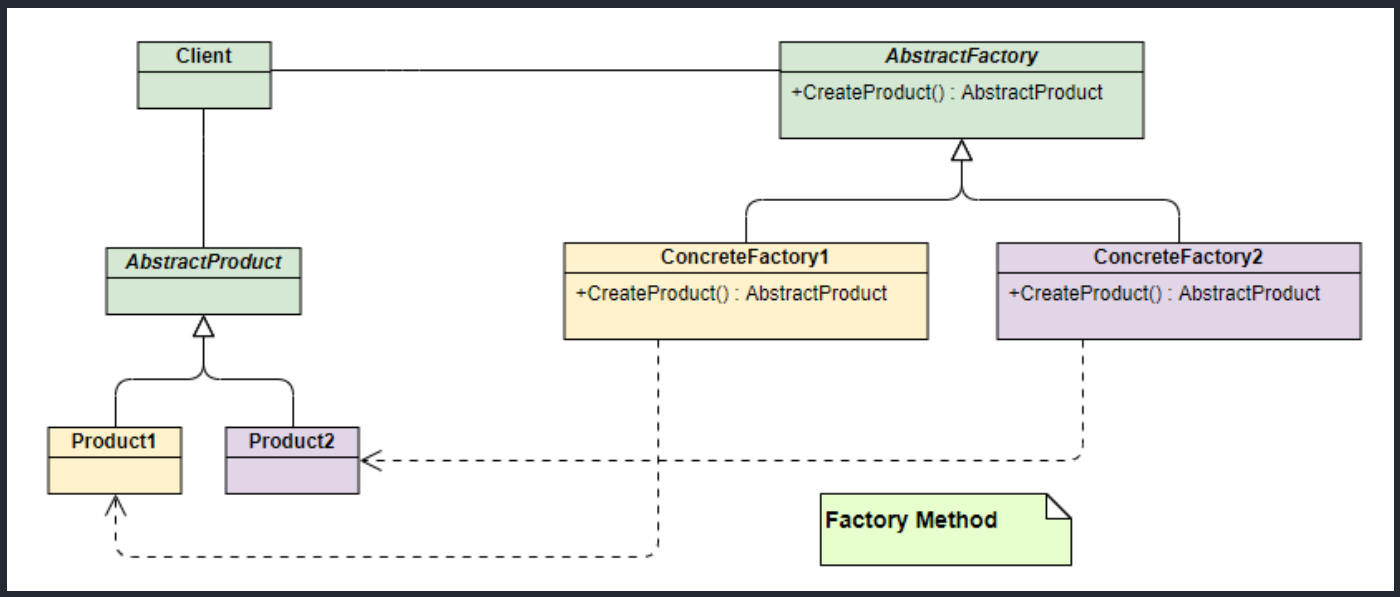
维度	静态工厂（Static Factory）	简单工厂（Simple Factory）
工厂位置	工厂方法以 `static` 方法形式内嵌在 **产品类或工具类** 中	工厂逻辑独立在 **单独的工厂类** 中
产品范围	通常用于创建**同一类产品**（或其变体）	用于创建**多个不同类型但相关的产品**（如同一接口的不同实现）
参数依赖	通常用**不同方法名**区分产品，可零参数	必须传**参数**（字符串/枚举等）指定产品类型
扩展性	新增产品需修改包含静态方法的类，违反 OCP	新增产品需修改工厂类判断逻辑，同样违反 OCP
核心优势	API 语义清晰（如 `LocalDate.of`），调用直观	创建逻辑集中管理，客户端无需关心具体实现

NIO 中大量用到了工厂模式，比如 `Files` 类的 `newInputStream` 方法用于创建 `InputStream` 对象（静态工厂）、 `Paths` 类的 `get` 方法创建 `Path` 对象（静态工厂）、 `ZipFileSystem` 类（ `sun.nio` 包下的类，属于 `java.nio` 相关的一些内部实现）的 `getPath` 的方法创建 `Path` 对象（简单工厂）。可以顺带阅读一下笔者写的这篇 Java IO 设计模式总结。

工厂方法模式了解吗?

为了解决简单工厂模式违反开闭原则的问题，工厂方法模式将创建具体对象的任务**下放给子类**。它定义了一个用于创建对象的抽象方法，但由子类来决定要实例化哪一个类。

UML 类图如下：



- **角色组成**：
  - **抽象工厂 (AbstractFactory)**：声明了创建产品的抽象方法。
  - **具体工厂 (ConcreteFactory)**：实现抽象方法，负责创建具体的产品。
  - **抽象产品 (AbstractProduct)**：与简单工厂模式相同。
  - **具体产品 (ConcreteProduct)**：与简单工厂模式相同。
- **解决的问题**：在不修改现有工厂类的情况下，轻松**扩展新的产品**。如果想增加一个新产品，只需增加一个新的具体产品类和一个对应的具体工厂类即可，完全符合**开闭原则**。
- **优缺点**：
  - **优点**：完美遵循开闭原则，扩展性好。创建逻辑被分散到各个具体工厂中，符合单一职责原则。
  - **缺点**：每增加一个产品，就需要增加一个具体工厂类，这会导致系统中的**类数量成倍增加**，增加了系统的复杂性。
- **实例**：Java 集合框架中 `Collection` 接口的 `iterator()` 方法。`ArrayList` 和 `LinkedList` 都实现了这个接口，但它们各自的 `iterator()` 方法返回的是不同的迭代器实现类（`ArrayListIterator` 和 `LinkedListIterator`），这就是典型的工厂方法模式。

工厂方法代码实现：

```
package facytoy;

// 抽象产品：电子设备
interface Device {
    void operate(); // 操作设备
```

```java
}

// 具体产品：手机
class Phone implements Device {
    @Override
    public void operate() {
        System.out.println("手机操作：开机 -> 使用 -> 关机\n");
    }
}

// 具体产品：电脑
class Computer implements Device {
    @Override
    public void operate() {
        System.out.println("电脑操作：开机 -> 使用 -> 关机\n");
    }
}

// 抽象工厂：设备工厂接口
interface DeviceFactory {
    Device createDevice();
}

// 具体工厂：手机工厂
class PhoneFactory implements DeviceFactory {
    @Override
    public Device createDevice() {
        System.out.println("手机工厂生产手机：");
        return new Phone();
    }
}

// 具体工厂：电脑工厂
class ComputerFactory implements DeviceFactory {
    @Override
    public Device createDevice() {
        System.out.println("电脑工厂生产电脑：");
        return new Computer();
    }
```

```
}

// 客户端使用
public class SimplifiedFactoryMethodDemo {
    public static void main(String[] args) {
        // 创建专门的工厂
        DeviceFactory phoneFactory = new PhoneFactory();
        DeviceFactory computerFactory = new ComputerFactory();

        // 生产并使用设备
        phoneFactory.createDevice().operate();
        computerFactory.createDevice().operate();
    }
}
```

输出结果：

```
手机工厂生产手机：
手机操作：开机 -> 使用 -> 关机

电脑工厂生产电脑：
电脑操作：开机 -> 使用 -> 关机
```

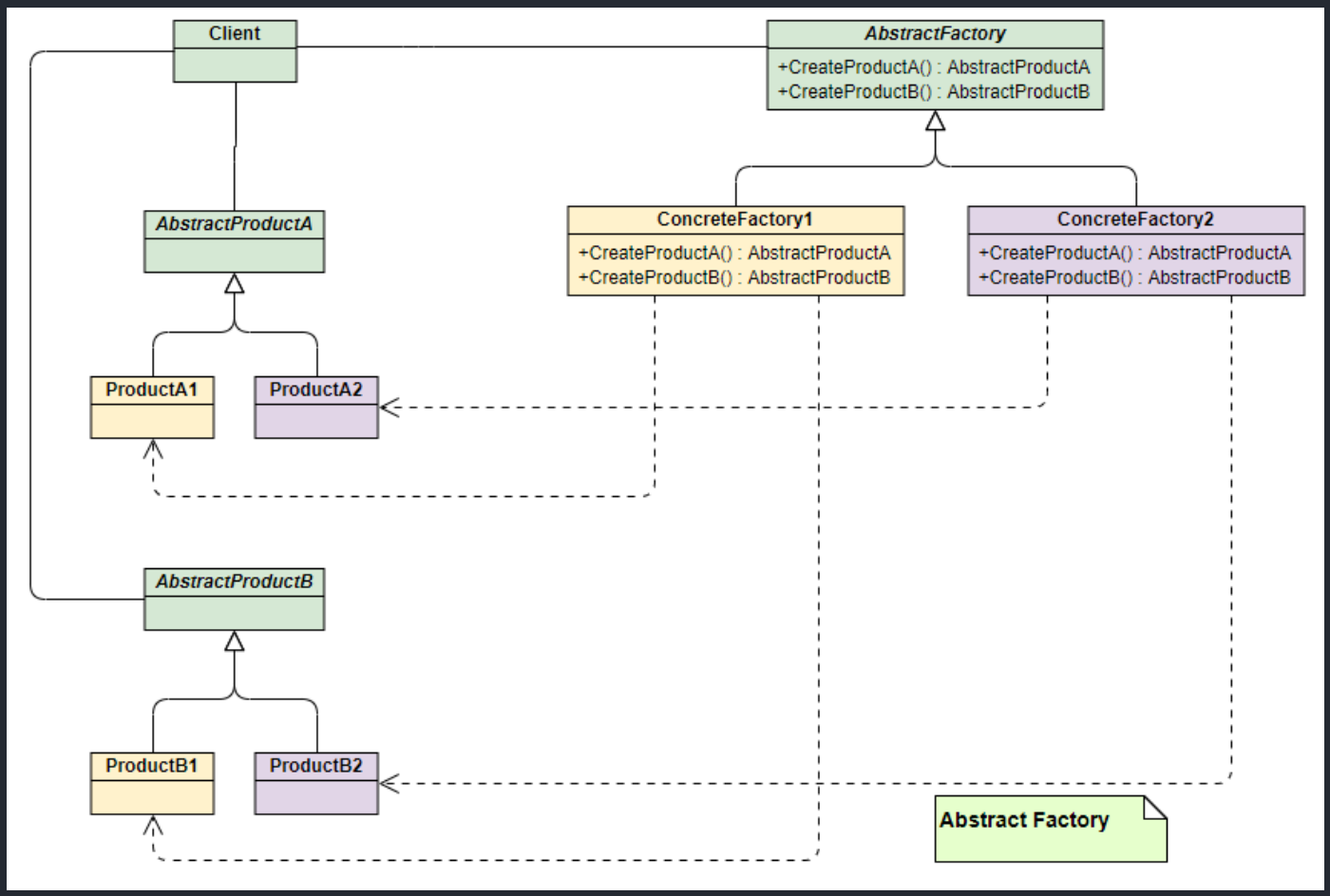
抽象工厂模式了解吗?

当需要创建的**不是单一产品，而是一个产品族（一系列相互关联或相互依赖的对象）**时，就该使用抽象工厂模式。它提供一个接口，用于创建一系列相关或相互依赖的对象，而无需指定它们具体的类。可以理解为它是“**工厂的工厂**”。

"产品族"和 "产品等级结构"概念介绍：

- 产品等级结构：同一类产品的不同实现（如按钮可以有 Windows 按钮、Mac 按钮，这构成一个产品等级结构）
- 产品族：同一品牌或风格下的不同产品（如 Windows 风格下的按钮、文本框、复选框，构成一个产品族）

UML 类图如下：



- **角色组成**：
  - **抽象工厂 (AbstractFactory)**：定义了创建一系列不同产品（一个产品族）的接口。
  - **具体工厂 (ConcreteFactory)**：实现接口，创建特定主题或风格的产品族。
  - **抽象产品 (AbstractProduct)**：为产品族中的每一种产品定义接口。
  - **具体产品 (ConcreteProduct)**：实现抽象产品接口，是具体工厂创建的目标。
- **解决的问题**：解决创建**一整个产品家族**的问题，保证客户端在切换不同产品族时，能获得一整套相互兼容和匹配的对象。
- **优缺点**：
  - **优点**：非常适合用于创建一系列相互匹配的产品。切换整个产品族非常方便，只需更换具体的工厂即可。
  - **缺点**：**扩展新的产品等级结构困难**。例如，如果产品族需要增加一个“鼠标”，那么 `AbstractFactory` 接口就需要增加一个 `createMouse()` 方法，所有已经实现的具体工厂类也都要跟着修改，这违反了开闭原则。
- **实例**：

- 最典型的例子是更换软件皮肤。一个 `SkinFactory` 接口可以定义 `createButton()` 、 `createTextBox()` 等方法。然后可以有 `WindowsSkinFactory` 和 `MacSkinFactory` 两个具体工厂，分别用来创建一套 Windows 风格或 Mac 风格的 UI 组件。
- JDBC 也是一个很好的例子。 `Connection` 、 `Statement` 、 `ResultSet` 就可以看作一个产品族。不同的数据库驱动（如 MySQL Driver、Oracle Driver）就扮演了具体工厂的角色，负责创建对应数据库的连接和操作对象。

抽象工厂代码实现：

```java
package facytoy;

// 抽象产品A：设备
interface Device {
    void operate(); // 操作设备
}

// 抽象产品B：配件
interface Accessory {
    void use(); // 使用配件
}

// 具体产品A1：智能手机
class SmartPhone implements Device {
    @Override
    public void operate() {
        System.out.println("智能手机：开机使用");
    }
}

// 具体产品A2：笔记本电脑
class Laptop implements Device {
    @Override
    public void operate() {
        System.out.println("笔记本电脑：开机使用");
    }
}

// 具体产品B1：手机壳
```

```java
class PhoneCase implements Accessory {
    @Override
    public void use() {
        System.out.println("手机壳: 保护手机\n");
    }
}

// 具体产品B2: 电脑包
class LaptopBag implements Accessory {
    @Override
    public void use() {
        System.out.println("电脑包: 携带电脑\n");
    }
}

// 抽象工厂: 设备套装工厂
interface DeviceSetFactory {
    Device createDevice();
    Accessory createAccessory();
}

// 具体工厂: 手机套装工厂
class PhoneSetFactory implements DeviceSetFactory {
    @Override
    public Device createDevice() {
        return new SmartPhone();
    }

    @Override
    public Accessory createAccessory() {
        return new PhoneCase();
    }
}

// 具体工厂: 电脑套装工厂
class LaptopSetFactory implements DeviceSetFactory {
    @Override
    public Device createDevice() {
        return new Laptop();
```

```java
    }

    @Override
    public Accessory createAccessory() {
        return new LaptopBag();
    }
}

// 客户端使用
public class SimplifiedAbstractFactoryDemo {
    public static void main(String[] args) {
        // 创建手机套装
        System.out.println("=== 手机套装 ===");
        DeviceSetFactory phoneFactory = new PhoneSetFactory();
        phoneFactory.createDevice().operate();
        phoneFactory.createAccessory().use();

        // 创建电脑套装
        System.out.println("=== 电脑套装 ===");
        DeviceSetFactory laptopFactory = new LaptopSetFactory();
        laptopFactory.createDevice().operate();
        laptopFactory.createAccessory().use();
    }
}
```

输出结果：

```
=== 手机套装 ===
智能手机：开机使用
手机壳：保护手机

=== 电脑套装 ===
笔记本电脑：开机使用
电脑包：携带电脑
```

简单工厂、工厂方法和抽象工厂模式的区别

特性	简单工厂模式	工厂方法模式	抽象工厂模式
复杂度	低	中	高
关注点	创建**单一类型**的产品	创建**单一类型**的产品	创建**一族相关联**的产品
开闭原则	**违反**	**遵守**	**对扩展产品族遵守，对扩展产品类型违反**
核心	一个集中式工厂	将创建逻辑延迟到子类	创建产品家族

单例模式

什么是单例模式？有什么优点？

单例模式属于创建型模式，一个单例类在任何情况下都只存在一个实例，构造方法必须是私有的、由自己创建一个静态变量存储实例，对外提供一个静态公有方法获取实例。

优缺点：

- **优点**：
  - **节省资源**：由于内存中只有一个实例，减少了频繁创建和销毁对象带来的性能开销。
  - **全局控制**：方便对唯一实例进行统一管理，避免对共享资源的多重占用。
- **缺点**：
  - **扩展性差**：因为没有抽象层，难以扩展。
  - **职责过重**：一个类既要负责自身的业务逻辑，又要负责保证单例，有点违背单一职责原则。
  - **测试困难**：全局状态在单元测试中可能引入依赖和副作用。

单例模式的常见写法有哪些？

单例模式的实现方式有很多种，它们主要在**线程安全**和**懒加载**（Lazy Loading）这两个维度上有所不同。

1. 饿汉式 (Eager Initialization) - 线程安全

饿汉式单例模式，顾名思义，类一加载就创建对象，这种方式比较常用，但容易产生垃圾对象，浪费内存空间。

饿汉式单例是如何保证线程安全的呢？它是基于类加载机制避免了多线程的同步问题，但是如果类被不同的类加载器加载就会创建不同的实例。

代码实现，以及使用反射破坏单例：

```
public class Singleton {
    // 1. 私有化构造
    private Singleton() {}
    // 2. 类加载时就创建实例
    private static final Singleton INSTANCE = new Singleton();
    // 3. 提供公共访问方法
    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

使用反射破坏单例，代码如下：

```
public class Test {

    public static void main(String[] args) throws Exception{
        // 使用反射破坏单例
        // 获取空参构造方法
        Constructor<Singleton> declaredConstructor =
Singleton.class.getDeclaredConstructor(null);
        // 设置强制访问
        declaredConstructor.setAccessible(true);
        // 创建实例
        Singleton singleton = declaredConstructor.newInstance();
```

```
        System.out.println("反射创建的实例" + singleton);
        System.out.println("正常创建的实例" + Singleton.getInstance());
        System.out.println("正常创建的实例" + Singleton.getInstance());
    }
}
```

输出结果如下：

```
反射创建的实例com.example.spring.demo.single.Singleton@6267c3bb
正常创建的实例com.example.spring.demo.single.Singleton@533ddba
正常创建的实例com.example.spring.demo.single.Singleton@533ddba
```

优缺点：

- **优点**：实现简单，天生线程安全，执行效率高。
- **缺点**：不是懒加载。如果这个实例从未使用过，会造成内存浪费。

> 懒加载 （lazy loading）：使用的时候再创建对象

2. 懒汉式 (Lazy Initialization) - 线程不安全

为了解决饿汉式的资源浪费问题，懒汉式在第一次被使用时才创建实例。但下面这种基础写法是**线程不安全**的。

```
public class Singleton  {
    // 1、私有化构造方法
    private Singleton(){ }
    // 2、定义一个静态变量指向自己类型
    private static Singleton instance;
    // 3、对外提供一个公共的方法获取实例
    public static Singleton getInstance() {
        // 判断为 null 的时候再创建对象
        if (instance == null) {
            instance = new Singleton();
```

```
        }
        return instance;
    }
}
```

使用多线程破坏单例，测试代码如下：

```
public class Test {

    public static void main(String[] args) {
        for (int i = 0; i < 3; i++) {
            new Thread(() -> {
                System.out.println("多线程创建的单例：" +
Singleton.getInstance());
            }).start();
        }
    }
}
```

输出结果如下：

```
多线程创建的单例：com.example.spring.demo.single.Singleton@18396bd5
多线程创建的单例：com.example.spring.demo.single.Singleton@7f23db98
多线程创建的单例：com.example.spring.demo.single.Singleton@5000d44
```

优缺点：

- **优点**：实现了懒加载。
- **缺点**：线程不安全。在多线程环境下，可能创建出多个实例。

3. 懒汉式 - 同步方法

懒汉式单例如何保证线程安全呢？ 通过 `synchronized` 关键字加锁保证线程安全，`synchronized` 可以添加在方法上面，也可以添加在代码块上面，这里演示添加在方法上面，存在的问题是每一次调用 `getInstance` 获取实例时都需要加锁和释放锁，这样是非常影响性能的。

```java
public class Singleton  {
    // 1、私有化构造方法
    private Singleton(){ }
    // 2、定义一个静态变量指向自己类型
    private static Singleton instance;
    // 3、对外提供一个公共的方法获取实例
    public synchronized static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

4. 懒汉式 - 双重检查锁

双重检查锁 (Double-Checked Locking, DCL) 是对同步方法懒汉式的优化，它通过两次 null 检查和 `synchronized` 代码块，在保证线程安全的同时，大大提升了性能。

```java
public class Singleton {
    private Singleton() {}
    // 使用 volatile 保证可见性和禁止指令重排
    private static volatile Singleton instance;

    public static Singleton getInstance() {
        // 第一次检查：避免不必要的同步
        if (instance == null) {
            // 第二次检查：在同步块内部确保只有一个线程创建实例
            synchronized (Singleton.class) {
                if (instance == null) {
                    // new 操作不是原子性的，需要 volatile
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
```

```
    }
```

双重检查的必要性：

- **外层检查**：当实例已创建时，所有线程可直接返回结果，避免不必要的同步开销，这是性能优化的关键。
- **内层检查**：解决并发竞争问题 —— 当多个线程同时通过外层检查时，同步块保证只有一个线程进入创建逻辑，后续线程会被内层检查拦截，确保仅创建一个实例。

volatile 关键字的作用：

`instance` 采用 `volatile` 关键字修饰也是很有必要的， `instance = new Singleton();` 这段代码其实是分为三步执行：

1. 为 `instance` 分配内存空间
2. 初始化 `instance`
3. 将 `instance` 指向分配的内存地址

但是由于 JVM 具有指令重排的特性，执行顺序有可能变成 1->3->2。指令重排在单线程环境下不会出现问题，但是在多线程环境下会导致一个线程获得还没有初始化的实例。例如，线程 T1 执行了 1 和 3，此时 T2 调用 `getInstance` () 后发现 `instance` 不为空，因此返回 `instance` ，但此时 `instance` 还未被初始化。

既然聊到了多线程，建议你顺带复习一下这块的知识点，推荐阅读笔者写的这几篇文章：

- Java并发常见面试题总结（上） （多线程基础知识，例如线程和进程的概念、死锁）
- Java并发常见面试题总结（中） （各种锁，例如乐观锁和悲观锁、 `synchronized` 关键字、 `ReentrantLock` ）
- Java并发常见面试题总结（下）(`ThreadLocal` 、线程池、 `Future` 、AQS、虚拟线程等)

优缺点：

- **优点**：懒加载、线程安全、性能较高。
- **缺点**：实现相对复杂，需要正确理解 volatile 的作用

5. 静态内部类 - 推荐

虽然 DCL 已经很优秀了，但在现代 Java 开发中，我们有更优雅、更推荐的实现方式。

这是我个人比较推荐的写法，它巧妙地利用了 **JVM 的类加载机制**来实现懒加载和线程安全。

```
public class Singleton {
    private Singleton() {}

    private static class SingletonHolder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return SingletonHolder.INSTANCE;
    }
}
```

静态内部类单例是如何实现懒加载的呢？首先，我们先了解下类的加载时机。

对于初始化阶段，虚拟机严格规范了有且仅有 6 种情况下，必须对类进行初始化(只有主动去使用类才会初始化类)：

1. 遇到 `new` 、 `getstatic` 、 `putstatic` 或 `invokestatic` 这 4 条字节码指令时：
    - `new` ：创建一个类的实例对象。
    - `getstatic` 、 `putstatic` ：读取或设置一个类型的静态字段（被 `final` 修饰、已在编译期把结果放入常量池的静态字段除外）。
    - `invokestatic` ：调用类的静态方法。
2. 使用 `java.lang.reflect` 包的方法对类进行反射调用时如 `Class.forName("...")` ， `newInstance()` 等等。如果类没初始化，需要触发其初始化。
3. 初始化一个类，如果其父类还未初始化，则先触发该父类的初始化。
4. 当虚拟机启动时，用户需要定义一个要执行的主类 (包含 `main` 方法的那个类)，虚拟机会先初始化这个类。
5. `MethodHandle` 和 `VarHandle` 可以看作是轻量级的反射调用机制，而要想使用这 2 个调用，就必须先使用 `findStaticVarHandle` 来初始化要调用的类。

6. **「补充，来自issue745」** 当一个接口中定义了 JDK8 新加入的默认方法（被 default 关键字修饰的接口方法）时，如果有这个接口的实现类发生了初始化，那该接口要在其之前被初始化。

关于类加载的详细介绍可以阅读笔者写的这篇文章：类加载过程详解。

这 6 种情况被称为是类的主动引用，注意，这里《虚拟机规范》中使用的限定词是 "**有且仅有**"，那么，除此之外的所有引用类都不会对类进行初始化，称为被动引用。静态内部类就属于被动引用的情况。

当 `getInstance()` 方法被调用时，`InnerClass` 才在 `Singleton` 的运行时常量池里，把符号引用替换为直接引用，这时静态对象 `INSTANCE` 也真正被创建，然后再被 `getInstance()` 方法返回出去，这点同饿汉模式。

那么 `INSTANCE` 在创建过程中又是如何保证线程安全的呢？在《深入理解 JAVA 虚拟机》中，有这么一句话：

> 虚拟机会保证一个类的 `<clinit>()` 方法在多线程环境中被正确地加锁、同步，如果多个线程同时去初始化一个类，那么只会有一个线程去执行这个类的 `<clinit>()` 方法，其他线程都需要阻塞等待，直到活动线程执行 `<clinit>()` 方法完毕。如果在一个类的 `<clinit>()` 方法中有耗时很长的操作，就可能造成多个进程阻塞(**需要注意的是，其他线程虽然会被阻塞，但如果执行 `<clinit>()` 方法后，其他线程唤醒之后不会再次进入 `<clinit>()` 方法。同一个加载器下，一个类型只会初始化一次。**)，在实际应用中，这种阻塞往往是很隐蔽的。

从上面的分析可以看出 `INSTANCE` 在创建过程中是线程安全的，所以说静态内部类形式的单例可保证线程安全，也能保证单例的唯一性，同时也延迟了单例的实例化。

6. 枚举单例 - 推荐

《Effective Java》作者 Joshua Bloch 极力推崇的方式。

```java
public enum Singleton {
    INSTANCE;

    public void doSomething() {
        System.out.println("...doing something");
    }
}
```

枚举在 java 中与普通类一样，都能拥有字段与方法，而且枚举实例创建是线程安全的，在任何情况下，它都是一个单例，可以直接通过如下方式调用获取实例：

```java
Singleton singleton = Singleton.INSTANCE;
```

使用下面的命令反编译枚举类

```
javap Singleton.class
```

得到如下内容

```
Compiled from "Singleton.java"
public final class com.spring.demo.singleton.Singleton extends
java.lang.Enum<com.spring.demo.singleton.Singleton> {
  public static final com.spring.demo.singleton.Singleton INSTANCE;
  public static com.spring.demo.singleton.Singleton[] values();
  public static com.spring.demo.singleton.Singleton valueOf(java.lang.String);
  public void doSomething(java.lang.String);
  static {};
}
```

从枚举的反编译结果可以看到，INSTANCE 被 `static final` 修饰，所以可以通过类名直接调用，**并且创建对象的实例是在静态代码块中创建的**，因为 static 类型的属性会在类被加载之后被初始化，当一个 Java 类第一次被真正使用到的时候静态资源被初始化、Java 类的加载和初始化过程都是线程安全的，所以创建一个 enum 类型是线程安全的。

通过反射破坏枚举，实现代码如下：

```
public class Test {
    public static void main(String[] args) throws Exception {
        Singleton singleton = Singleton.INSTANCE;
        singleton.doSomething("hello enum");

        // 尝试使用反射破坏单例
        // 枚举类没有空参构造方法，反编译后可以看到枚举有一个两个参数的构造方法
        Constructor<Singleton> declaredConstructor =
Singleton.class.getDeclaredConstructor(String.class, int.class);
        // 设置强制访问
        declaredConstructor.setAccessible(true);
        // 创建实例，这里会报错，因为无法通过反射创建枚举的实例
        Singleton enumSingleton = declaredConstructor.newInstance();
        System.out.println(enumSingleton);
    }
}
```

运行结果报如下错误：

```
Exception in thread "main" java.lang.IllegalArgumentException: Cannot
reflectively create enum objects
    at
java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:
492)
    at
java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:480)
    at com.spring.demo.singleton.Test.main(Test.java:24)
```

查看反射创建实例的 `newInstance()` 方法，有如下判断：

```java
Constructor.java
@NotNull
@CallerSensitive
@ForceInline // to ensure Reflection.getCallerClass optimization
public T newInstance(Object ... initargs)
    throws InstantiationException, IllegalAccessException,
           IllegalArgumentException, InvocationTargetException
{
    Class<?> caller = override ? null : Reflection.getCallerClass();
    return newInstanceWithCaller(initargs, !override, caller);
}

/* package-private */
T newInstanceWithCaller(Object[] args, boolean checkAccess, Class<?> caller)
    throws InstantiationException, IllegalAccessException,
           InvocationTargetException
{
    if (checkAccess)
        checkAccess(caller, clazz, clazz, modifiers);

    if ((clazz.getModifiers() & Modifier.ENUM) != 0)
        throw new IllegalArgumentException("Cannot reflectively create enum objects");

    ConstructorAccessor ca = constructorAccessor;   // read volatile
```

所以无法通过反射创建枚举的实例。

- **优点**：
  - 实现极其简单。
  - 天生线程安全，由 JVM 从语言层面保证。
  - 能有效防止通过反射和反序列化来破坏单例。Java 规定，不能通过反射来创建枚举实例，并且在反序列化时，JVM 会特殊处理，保证返回的是同一个枚举实例。
- **缺点**：不是懒加载。

适配器模式

适配器模式了解吗？

适配器模式（Adapter Pattern）是一种结构型设计模式，它的核心作用是**将一个类的接口转换成客户端所期望的另一个接口**，从而使原本因接口不兼容而不能在一起工作的类可以协同工作。

你可以把它想象成一个我们日常生活中常用的**电源适配器或转换头**。比如，你的笔记本电脑电源是两脚插头（被适配者 Adaptee），但墙上的插座是三孔的（目标 Target）。这时，你需要一个转换头（适配器 Adapter），它一端能插进三孔插座，另一端能接收你的两脚插头，这样问题就解决了。

适配器模式主要包含三个核心角色：

- **Target (目标接口)**：客户端（Client）期望和它直接交互的接口。在上面的例子里，就是那个三孔插座。
- **Adaptee (被适配者)**：已存在的、但接口与 `Target` 不兼容的类。也就是那个两脚插头。
- **Adapter (适配器)**：模式的核心。它实现了 `Target` 接口，同时内部包装了一个 `Adaptee` 类的实例，负责将对 `Target` 接口的调用转换为对 `Adaptee` 接口的调用。它就是那个转换头。

优缺点：

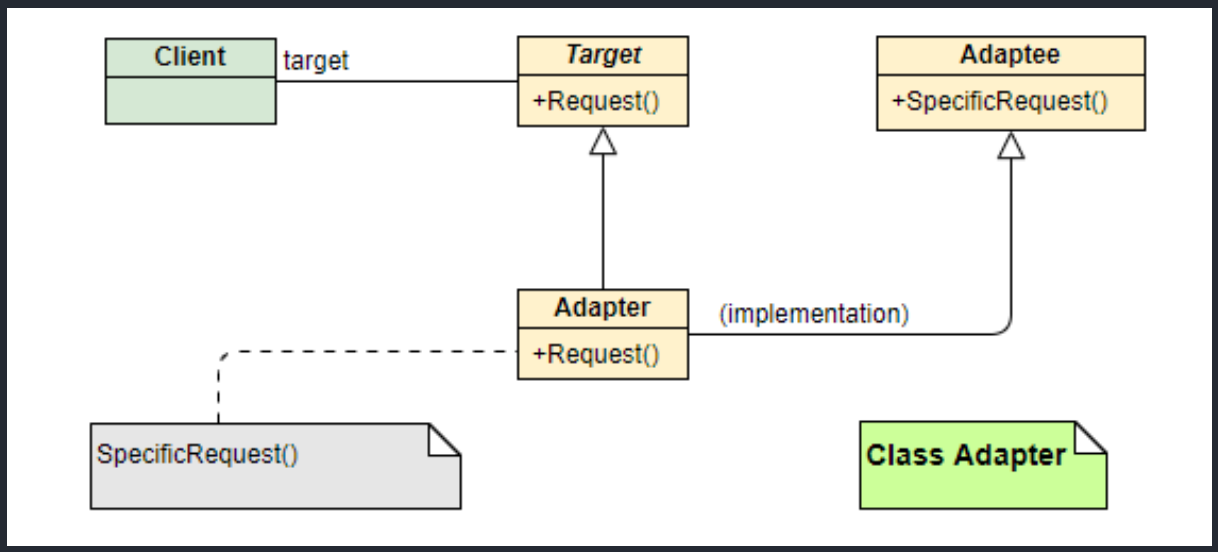
- **优点：**
  - **增强了类的复用性**：可以复用已存在的、功能强大的 `Adaptee` 类，而无需修改其源码。
  - **提高了灵活性和扩展性**：可以非常方便地替换或增加新的适配器，来适配不同的 `Adaptee`，符合开闭原则。
  - **解耦**：将客户端（Client）与具体的实现类（Adaptee）解耦，客户端只需要和目标接口（Target）打交道。
- **缺点：**
  - **增加了系统的复杂性**：每适配一个类都需要增加一个适配器类，如果过度使用，会导致系统中的类数量增多，代码可读性有所下降。
  - （针对类适配器）**限制较多**：由于语言的单继承限制，类适配器一次最多只能适配一个 `Adaptee` 类，并且要求 `Target` 必须是接口或抽象类。

适配器模式实现方式有哪些？如何选择？

适配器模式主要有两种实现方式：**类适配器**和**对象适配器**。

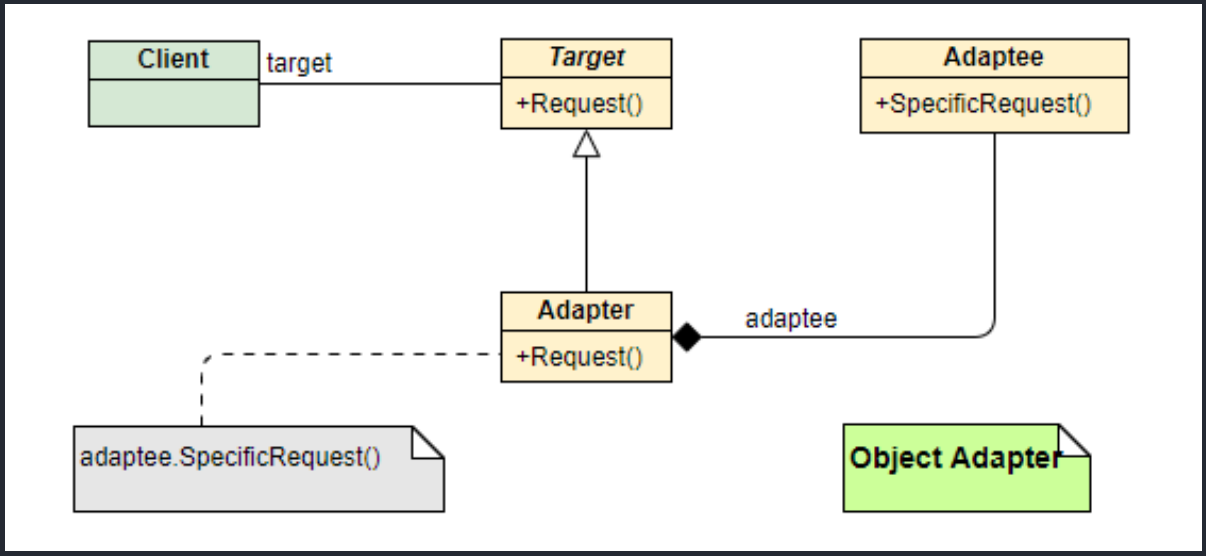
类适配器：

- **原理**：通过**类继承**来实现。`Adapter` 类同时继承 `Adaptee` 类并实现 `Target` 接口。
- **特点**：由于 Java 是单继承，这意味着 `Adapter` 只能适配一个 `Adaptee` 类。它的耦合度相对较高。



对象适配器：

- **原理**：通过**对象组合/关联**来实现。`Adapter` 类实现 `Target` 接口，并在内部持有一个 `Adaptee` 类的实例。
- **特点**：这种方式更加灵活，因为 `Adapter` 可以适配 `Adaptee` 的任何子类。它遵循了“合成/聚合复用原则”，是更推荐、更常用



简单对比一下：

对比维度	类适配器	对象适配器
实现原理	继承	组合/关联
耦合度	高（编译时绑定）	低（运行时绑定）
灵活性	低，只能适配一个Adaptee类	高，可以适配Adaptee及其所有子类
推荐度	一般	**推荐**

哪些地方用到了适配器模式?

- `java.util.Arrays.asList()`：这个方法就是一个典型的适配器。它把一个数组（`Adaptee`）适配成一个`List`接口（`Target`），让我们可以用操作`List`的方式去操作一个数组。
- **`java.io`中的字符流与字节流转换**: `InputStreamReader`就是一个适配器。它将一个字节输入流`InputStream`（`Adaptee`）适配成一个字符输入流`Reader`（`Target`），解决了处理文本文件时字节到字符的转换问题。`OutputStreamWriter`也是同理。可以顺带阅读一下笔者写的这篇Java IO 设计模式总结。
- **日志框架 SLF4J**: SLF4J (Simple Logging Facade for Java) 本身是一个日志门面，但它的桥接包（如 slf4j-log4j12）就是适配器。它让你的应用程序代码（Client）统一面向 SLF4J 的 API（`Target`）编程，而底层可以无缝地切换到 Log4j、Logback 等具体的日志实现（`Adaptee`）。

代理模式

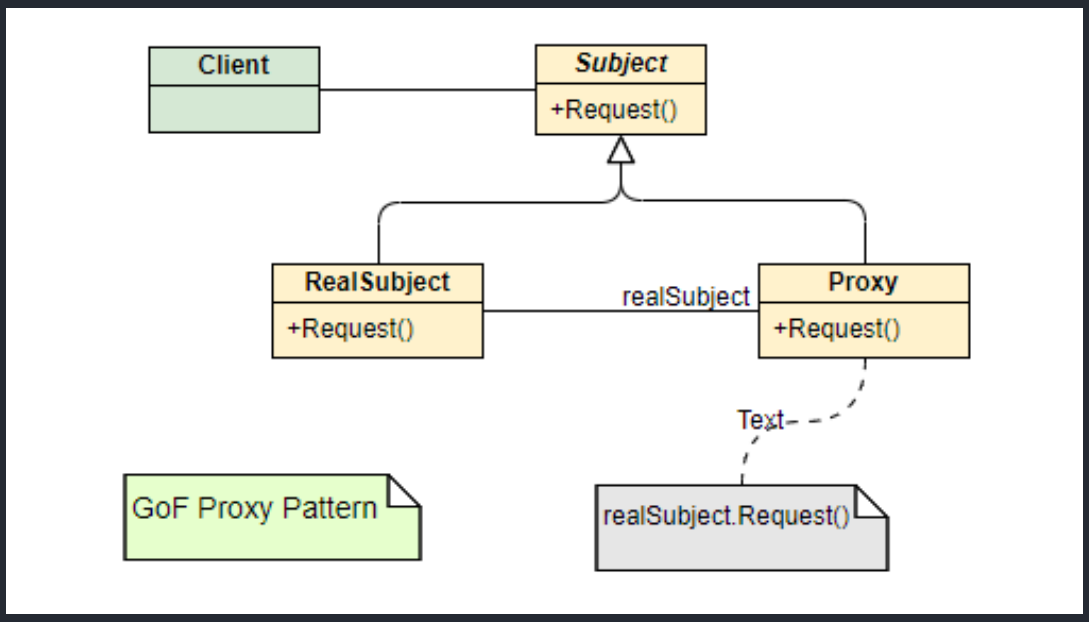
什么是代理模式?

代理模式（Proxy Pattern）是一种结构型设计模式。它的核心思想是**为其他对象提供一种代理，以控制对这个对象的访问。**

你可以把它想象成生活中的“明星经纪人”。粉丝（客户端 Client）不能直接联系到明星（真实对象 RealSubject），而是需要通过经纪人（代理 Proxy）。这个经纪人可以帮明星处理很多事情，比如过滤掉不重要的请求、安排日程、谈合同等等，而明星本人则可以专注于自己的核心工作——表演。

在这个过程中，经纪人和明星都实现了相同的“能力接口”（比如“接受采访”、“商业演出”），所以对外界来说，与经纪人沟通和与明星直接沟通，在接口上是一致的。

UML 类图：



代理模式主要包含三个核心角色：

- **Subject (抽象主题)**：定义了真实对象和代理对象共同的接口。
- **RealSubject (真实主题)**：被代理的实际对象，它执行业务的核心逻辑。
- **Proxy (代理)**：持有对真实对象的引用，并实现了抽象主题接口。它可以在调用真实对象前后执行额外的操作。这就是“经纪人”。

为什么要使用代理模式?

代理模式的价值在于，它可以在**不改变真实对象代码**的前提下，为其增加额外的功能。这些功能通常是通用的、与核心业务无关的，例如：

1. **远程代理 (Remote Proxy)**：为一个位于不同地址空间的对象提供本地的代表。它可以隐藏网络通信的细节，使得客户端调用远程对象就像调用本地对象一样。例如，Dubbo、gRPC 等 RPC 框架的客户端存根（Stub）就是典型的远程代理。
2. **虚拟代理 (Virtual Proxy)**：根据需要创建开销很大的对象。如果真实对象创建和初始化非常耗时，虚拟代理可以延迟它的创建，直到客户端真正需要它时才实例化。比如，加载一个高清大图，可以先显示一个占位符（代理），在后台真正加载图片（真实对象）。
3. **保护代理/安全代理 (Protection Proxy)**：控制对真实对象的访问权限。代理可以根据调用者的权限，决定是否将请求转发给真实对象。例如，Spring Security 中实现的方法级别的权限控制。
4. **智能引用 (Smart Reference)**：在访问对象时执行一些附加操作，如缓存、日志记录、事务管理等。这在 Spring AOP 中体现得淋漓尽致。

代理模式在 Java 中如何实现?

关于静态代理和动态代理的具体实现和区别可以参考笔者写的这篇文章：Java 代理模式详解。

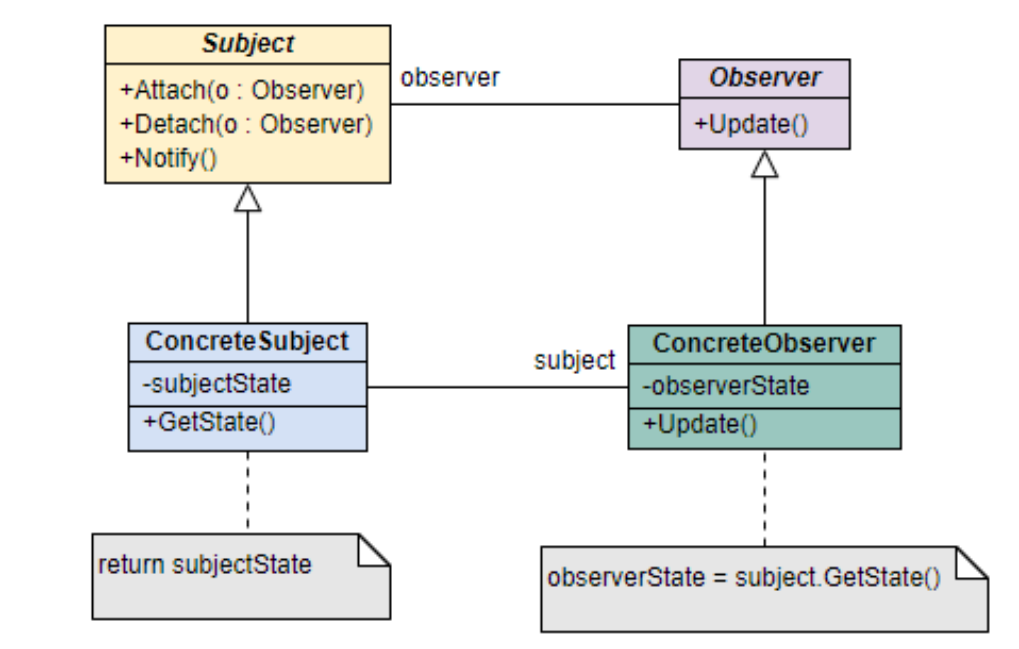
观察者模式

说一说观察者模式

观察者模式是一种非常经典和实用的**行为型设计模式**。它的核心思想在于定义了一种**一对多**的依赖关系：当一个对象（我们称之为“被观察者”或“主题”）的状态发生改变时，所有依赖于它的对象（即“观察者”）都会自动收到通知并进行相应的更新。

这种模式的本质是**解耦**，它将被观察者和观察者分离开来，使得它们可以独立地变化和复用，而不需要知道对方的具体实现细节。

UML 类图：



观察者模式通常包含四个核心角色：

- **主题/被观察者 (Subject):** 这是一个接口或抽象类，它负责维护一个观察者列表，并提供添加、删除观察者的接口。最关键的是，它还定义了通知所有观察者的 notify() 方法。
- **具体主题/具体被观察者 (ConcreteSubject):** 它是Subject的具体实现。它包含了业务逻辑，并在自身状态发生变化时，调用继承自Subject的 notify() 方法，通知所有已注册的观察者。

- **观察者 (Observer):** 同样是一个接口或抽象类，它定义了一个 update() 方法。当观察者接收到来自主题的通知时，这个方法就会被调用。
- **具体观察者 (ConcreteObserver):** 它是Observer的具体实现。在 update() 方法中，它会根据收到的通知，完成具体的业务逻辑，比如更新自身状态、执行某个操作等。

优缺点：

- **优点：**
  - **高度解耦：** 这是它最大的优点。主题和观察者之间是松耦合的，主题只知道它有一系列观察者，但不需要知道它们是谁、做什么。这为系统带来了极大的灵活性。
  - **符合开闭原则：** 系统是“对扩展开放，对修改关闭”的。当需要增加新的响应行为时，我们只需要创建一个新的具体观察者类并注册它即可，完全不需要修改主题的代码。
  - **支持广播通信：** 主题可以向所有注册的观察者广播通知，这在很多场景下非常高效。
- **缺点：**
  1. **潜在的性能问题：** 如果观察者数量非常多，或者某个观察者的 `update` 方法逻辑复杂、耗时较长，那么通知过程可能会导致主线程阻塞，影响性能。在某些场景下，可以考虑使用异步通知来优化。
  2. **可能导致意外的级联或循环：** 如果观察者之间存在复杂的依赖关系，一个观察者的更新可能会触发另一个观察者（它同时也是另一个主题）的更新，形成复杂的调用链。如果设计不当，甚至可能导致循环调用，造成系统崩溃。
  3. **调试困难：** 由于其松耦合的特性，当程序出现问题时，有时很难追踪到一个事件发生后，具体是哪个观察者的行为导致了问题。

补充一下：观察者模式在不同框架中可能有不同的命名风格，例如：

- Java 中的 `Observable`（主题）和 `Observer`（观察者）；
- Spring 中的 `ApplicationEvent`（事件，对应主题状态）和 `ApplicationListener`（监听器，对应观察者）。

你的项目是怎么用的观察者模式?

在我的项目中，观察者模式得到了广泛应用，一个非常典型的例子就是**支付成功后的业务处理**。

场景是这样的： 在一个电商系统中，当用户支付成功后，我们需要触发一系列独立的后续操作，比如：

- 更新订单状态为“已支付”。
- 给用户的账户增加积分。
- 通知仓储系统准备发货。
- 给用户发送一封确认邮件。

如果不用设计模式， 我们可能会在支付成功的方法里，把这四个操作串行地写下来。这样做的问题显而易见：支付核心逻辑与各种业务逻辑紧紧地耦合在一起，每次新增一个类似“赠送优惠券”的需求，都必须去修改这个已经很庞大和脆弱的核心方法，这严重违反了开闭原则。

我们的解决方案是：

我们将“支付成功”这个事件抽象为**具体主题 (ConcreteSubject)**。而上述的“更新订单”、“增加积分”、“通知仓库”、“发送邮件”等操作，则分别被设计成独立的**具体观察者 (** `ConcreteObserver` **)**。

工作流程是：

在系统初始化时，这些观察者会把自己注册到“支付成功”这个主题上。当支付网关回调我们的接口，确认支付成功后，主题的 `setState()` 方法被调用，然后它会立即执行 `notify()` ，遍历内部的观察者列表，并依次调用它们的 `update()` 方法。

这样做的价值在于：

- **高度解耦：** 支付核心流程完全不关心后续有哪些业务，它只负责发布“我成功了”这个消息。
- **极强的扩展性：** 未来如果需要增加“支付成功后赠送一张抽奖券”的新功能，我们只需要新增一个 `LotteryObserver` 类，并在系统启动时完成注册即可，原有代码一行都不用动。

装饰器模式

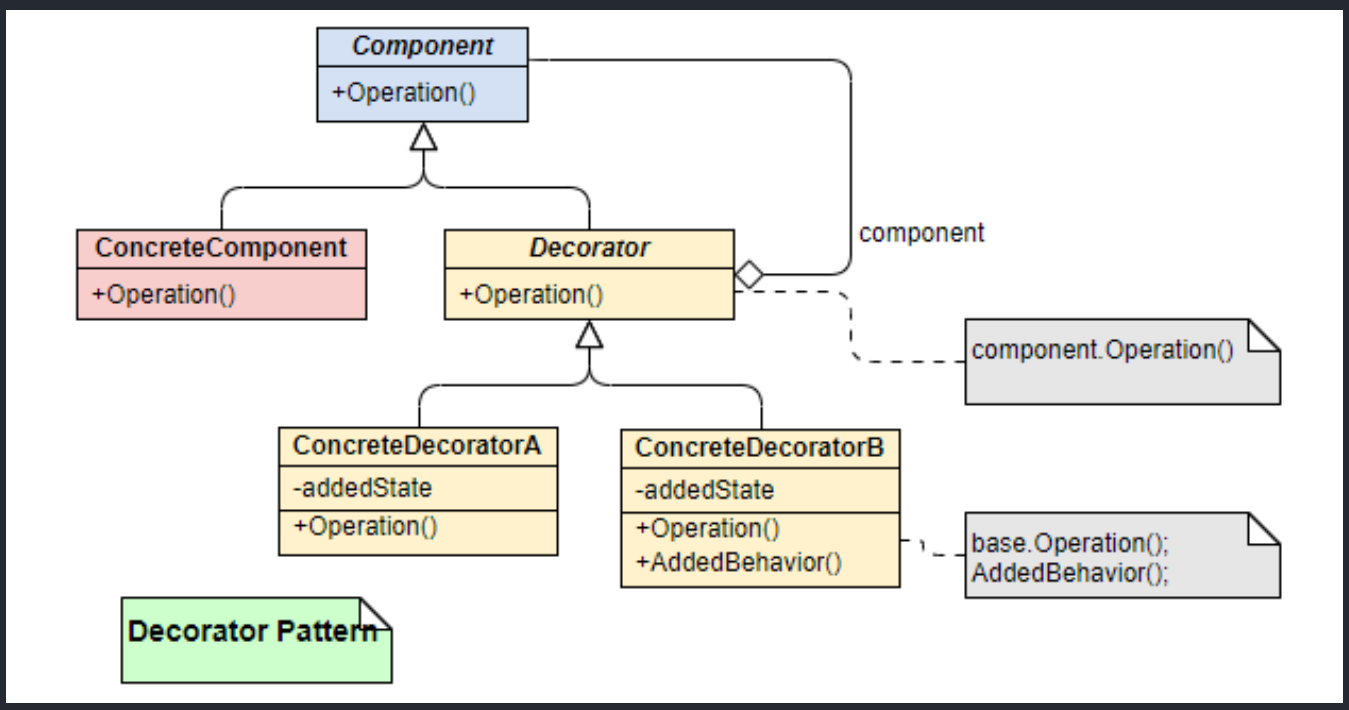
什么是装饰器模式?

装饰器模式是一种**结构型模式**，它的核心思想是**在不改变原有对象结构和代码的前提下，动态地为该对象添加额外的功能**。

它就像给一个物体“穿衣服”，你可以一层一层地套上不同的衣服（装饰器），每一件衣服都增加一种新的特性（功能），但物体的本质（核心功能）并没有改变。这个过程是通过创建一个包裹原始对象的“装饰器”对象来实现的，而不是通过继承。

装饰器模式是开闭原则的典范应用，它让我们在不修改既有代码的情况下，为对象赋予了无限的扩展可能。

UML 类图：



装饰器模式通常包含四个核心角色：

- **Component (抽象构件):** 这是一个接口或抽象类，它定义了原始对象和装饰器对象所共有的接口，确保它们可以被一致地对待。
- **ConcreteComponent (具体构件):** 这就是我们的“裸对象”，即被装饰的原始对象，它实现了 `Component` 接口。
- **Decorator (抽象装饰器):** 它也实现了 `Component` 接口，并且内部持有一个 `Component` 类型的引用（即它要装饰的对象）。它的存在是为了将装饰逻辑与基础组件分离开。
- **ConcreteDecorator (具体装饰器):** 这是实际的“衣服”。它继承自 `Decorator`，负责向被装饰的对象添加具体的新功能。在实现 `Component` 接口的方法时，它除了调用原始对象（通过持有的引用）的相应方法外，还会加上自己的“装饰”逻辑。

讲讲装饰器模式的应用场景

- **场景一：运行时动态扩展功能。** 当我们希望在运行时根据不同条件为对象增添不同功能时，装饰器模式是绝佳选择。最经典的例子就是Java的I/O流。 `FileInputStream` 是一个具体构件，我们可以用 `BufferedInputStream` 来装饰它，为其增加缓冲功能以提高性能；还可以再用 `DataInputStream` 来装饰，进一步为其增加读取基本数据类型的能力。这些装饰器可以任意组

合，非常灵活。可以顺带阅读一下笔者写的这篇 Java IO 设计模式总结。

- **场景二：当继承方案不可行或不适用时。** 如果一个类被 `final` 修饰，我们无法通过继承来扩展它。此时，装饰器模式就成了“救星”。此外，如果功能的组合种类非常多，使用继承会产生大量的子类，导致“类爆炸”问题，而装饰器模式则可以用少量的类灵活组合出所有需要的功能。

责任链模式

什么是责任链模式?

责任链模式（Chain of Responsibility Pattern）是一种**行为型设计模式**。它将请求的发送者和接收者解耦，通过创建一个处理请求的接收者链来处理请求。链上的每个接收者（也称为处理器或节点）都负责对请求进行一部分的处理或校验，并决定是否将请求传递给链上的下一个接收者，或者中断处理流程。

举个例子，你提交了一个电商订单，这个订单需要经过多个步骤的校验才能完成：库存校验 --> 风控校验 --> 支付信息校验 --> ... --> 订单完成 。每个校验步骤都像链条上的一个环节，只有通过当前环节的校验，订单才能进入下一个环节。任何一个环节校验失败，整个订单流程都会终止。

责任链模式的结构相对简单：

- **Handler (处理器):** 这是一个接口或抽象类，定义了处理请求的接口（如 `handleRequest()` ），以及一个指向下一个处理器的引用（ `setNext()` / `getNext()` ）。
- **ConcreteHandler (具体处理器):** 它实现了 `Handler` 接口。在处理方法中，它首先判断自己是否能处理当前请求。如果能，就处理它；如果不能，就将请求传递给链上的下一个处理器。

讲讲责任链模式的应用场景

适用于多节点的流程处理，每个节点完成各自负责的部分，节点之间不知道彼此的存在，比如：

- **订单校验：** 一个订单可能需要进行多种校验，例如商品库存校验、风控校验、支付信息校验等。可以将这些校验规则组成一个责任链，每个校验规则负责一种校验，如果校验不通过，则中断流程并返回错误信息；如果校验通过，则将请求传递给下一个校验规则。
- **OA的审批流：** 不同的审批级别（例如部门经理、总经理等）组成一个责任链。每个审批级别都负责审批一部分内容，如果审批不通过，则中断流程并返回原因；如果审批通过，则将请求传递给下一个审批级别。
- **Filter（过滤器）：** 多个Filter组成一个责任链，实现对 HTTP 请求的过滤功能，比如鉴权、限流、记录日志、验证参数等等。

- **Interceptor（拦截器）：** 类似于Filter，Interceptor也可以拦截请求并在请求处理前后执行一些操作。

策略模式

什么是策略模式?

策略模式的核心思想是：**定义一系列算法，将每一个算法都封装起来，并使它们可以相互替换。**

换句话说，它允许我们在运行时，根据不同的情况，动态地改变一个对象所使用的算法。这个对象（我们称之为“上下文/Context”）只依赖于一个抽象的策略接口，而不需要关心具体是哪种算法在执行。

这就像我们开车去一个地方，可以有好几种导航策略：比如“时间最短策略”、“距离最短策略”、“避开高速策略”。我们的“出行”这个行为（Context）本身不变，但可以随时切换具体使用的导航算法（Strategy）。

策略模式有什么好处?

1. **完美遵循开闭原则：** 这是策略模式最大的价值。当我们需要增加一种新的算法（比如新的“节假日促销策略”）时，我们只需要新增一个策略类即可，完全不需要修改使用这些策略的上下文（Context）代码。这使得系统扩展性极强。
2. **算法的独立与解耦：** 每个算法都被封装在独立的策略类中，使得算法本身可以独立于使用它的客户端而变化。这让代码更清晰，也更容易对每个算法进行单独的测试和维护。
3. **避免冗长的条件语句：** 它将复杂的、与业务逻辑紧密相关的 if-else 或 `switch-case` 结构，转变为一系列清晰、独立的策略类，大大提高了代码的可读性和可维护性。

谈谈你的项目策略模式的应用

这里我们以电商系统中的两个经典场景为例。

1. 电商系统的促销活动

`PromotionStrategy` 接口为策略接口，其中的核心方法 `calculate(double originalPrice)`，用于计算优惠后的价格。

针对不同优惠规则，实现策略接口：

- `CashRebateStrategy`（满减策略）：如“满 300 减 50”，在 `calculate` 中判断原价是否达标，返回减免后的价格。
- `DiscountStrategy`（折扣策略）：如“全场九折”，直接按比例计算折后价。
- `NoPromotionStrategy`（无优惠策略）：作为默认策略，直接返回原价。

在创建订单时，我们可以根据活动情况，向 `Order` 对象注入不同的促销策略实例。比如：

```
// 618活动：满300减50
PromotionStrategy june18Strategy = new CashRebateStrategy(300, 50);
Order order618 = new Order(june18Strategy);
double price618 = order618.getFinalPrice(400); // 结果：350

// 店庆活动：全场九折
PromotionStrategy anniversaryStrategy = new DiscountStrategy(0.9);
Order anniversaryOrder = new Order(anniversaryStrategy);
double priceAnniversary = anniversaryOrder.getFinalPrice(400); // 结果：360

// 无活动时：默认无优惠
Order normalOrder = new Order(new NoPromotionStrategy());
```

这样，新增优惠规则（如“第二件半价”）时，只需新增一个 `SecondHalfPriceStrategy` 实现类，无需修改 `Order` 或其他策略类，符合开闭原则。并且，优惠规则的修改（如满减门槛从 300 调整为 500）仅需调整对应策略的参数，不影响其他业务逻辑。

2. 支付方式选择

`PaymentStrategy` 接口为策略接口，其中的核心方法 `pay(amount)`，用于支付。

针对不同支付方式，实现策略接口：

- `AliPayStrategy`：支付宝支付。
- `WeChatPayStrategy`：微信支付。
- `CardPayStrategy`：银行卡支付。

用户选择不同的支付方式，`PaymentService` 就使用对应的策略对象来完成支付。

```java
public class PaymentService {
    private PaymentStrategy paymentStrategy;

    // 动态设置支付策略（如用户选择微信支付时传入WeChatPayStrategy）
    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.paymentStrategy = strategy;
    }

    public PaymentResult processPayment(double amount) {
        return paymentStrategy.pay(amount);
    }
}
```

这样，支付渠道的新增 / 下线仅需修改对应策略类，例如接入 “银联支付” 时，新增 `UnionPayStrategy` 即可，无需改动 `PaymentService` 或其他支付逻辑。并且，可以单独测试每种支付方式的异常场景（如支付超时、签名失败），提高代码的可维护性。

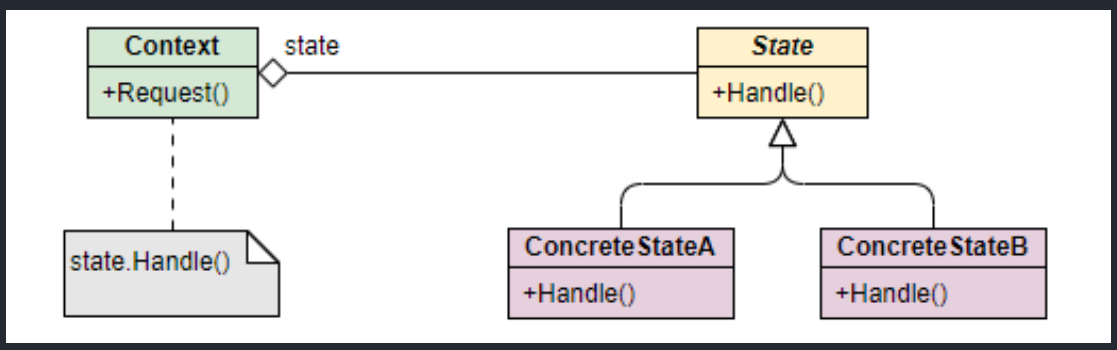
状态模式

什么是状态模式?

状态模式（State Pattern）是一种行为型设计模式。**它的核心思想是，允许一个对象的行为在内部状态改变时随之改变，这个对象看起来就像是修改了它的类。**

简单来说，状态模式将与特定状态相关的行为局部化，并将不同状态的行为分割到不同的状态类中。Context（上下文）对象将行为委托给当前的状态对象，从而消除了原来 Context 中庞大的条件分支语句（ `if-else` 或 `switch-case` ）。

UML 类图：



状态模式主要包含三个核心角色：

- **Context（上下文）**：维护一个 `State` 对象的实例，这个实例定义了对象的当前状态。它将与状态相关的请求委托给当前的状态对象处理。
- **State（抽象状态）**：定义一个接口，用于封装与 `Context` 的一个特定状态相关的行为。
- **ConcreteState（具体状态）**：实现 `State` 接口，每一个具体状态类都实现了该状态下的行为，并且负责在适当的时机进行状态的切换。

正如阿里巴巴《Java开发手册》中建议的：

> 超过3层的 if-else 的逻辑判断代码可以使用卫语句、策略模式、状态模式来实现。

7.【推荐】表达异常的分支时，少用 if-else 方式，这种方式可以改写成：

```
if (condition) {
    ...
    return obj;
}
// 接着写 else 的业务逻辑代码;
```

说明：如果非使用 if()...else if()...else...方式表达逻辑，避免后续代码维护困难，请勿超过 3 层。

正例：超过 3 层的 if-else 的逻辑判断代码可以使用卫语句、策略模式、状态模式等来实现，其中卫语句示例如下：

```
public void findBoyfriend(Man man) {
    if (man.isUgly()) {
        System.out.println("本姑娘是外貌协会的资深会员");
        return;
    }
```



```
    if (man.isPoor()) {
        System.out.println("贫贱夫妻百事哀");
        return;
    }
    if (man.isBadTemper()) {
        System.out.println("银河有多远，你就给我滚多远");
        return;
    }
    System.out.println("可以先交往一段时间看看");
}
```

优缺点：

- **优点：**
  - **结构清晰：** 将不同状态的行为封装到各自的类中，使得代码结构更清晰，职责更单一。
  - **易于扩展：** 增加新的状态非常方便，只需增加一个新的具体状态类即可，符合“开闭原则”。
  - **封装转换规则：** 状态之间的转换逻辑可以被封装在状态类内部，减少了上下文类的负担。
  - **避免条件滥用：** 有效避免了冗长的 if-else 或 switch 语句，提高了代码的可维护性。
- **缺点：**
  - **类数量增多：** 每个状态都需要一个对应的类，当状态过多时，会导致系统中类的数量增加。
  - **实现相对复杂：** 相比于简单的条件判断，状态模式的结构和实现逻辑更复杂。

- **状态转换耦合**：如果状态转换逻辑分散在各个具体状态类中，那么在增加新状态时，可能需要修改已有状态类的转换逻辑，这在一定程度上违反了“开闭原则”。

状态模式与策略模式的区别

状态模式和策略模式在结构上非常相似（都有一个上下文类、一个抽象接口和多个具体实现类），但它们的**意图**完全不同：

- **状态模式**：关注的是**状态的自动转换**。状态的变更通常由状态内部逻辑决定，对客户端是透明的。它旨在解决对象在不同状态下行为不同的问题，强调的是“下一个状态是什么”。
- **策略模式**：关注的是**策略的灵活选择**。具体使用哪种策略由客户端（或上层逻辑）决定并主动设置，策略之间通常没有直接关联。它旨在解决同一行为有不同实现算法的问题，强调的是“如何做”。

状态机了解吗?

状态机（State Machine），全称为有限状态自动机（Finite State Automaton），是一个用来描述系统在特定输入信号或事件下如何从一个状态转换到另一个状态的数学模型。在任何特定时间点，系统都处于某一确定的状态，而状态机定义了这些状态之间的转换规则。

状态机可以被视为一种设计模式，通常称为**状态模式（State Pattern）**。

状态机的引入可以使业务模型更加清晰，帮助开发人员更好地理解和实现业务逻辑。此外，状态机的使用有利于代码的维护和扩展。然而，状态机的引入也可能增加编码复杂性，对开发人员的编程技能提出了更高的要求。

状态机的四大概念：

1. **状态（State）**：状态机至少包含两个状态。例如，一个订单的状态可能包括“待支付”、“待发货”、“待收货”、“待评价”等。
2. **事件（Event）**：事件是触发状态转换的条件或指令。对于订单系统，事件可以是“支付成功”、“发货”、“确认收货”、“完成评价”等。
3. **动作（Action）**：在事件发生后将执行的动作，通常是调用某些方法。例如，在客户确认收货后，系统需执行更新订单状态到“待评价”的操作。
4. **转换（Transition）**：转换描述了一个状态如何基于某个事件转变到另一个状态。例如，当“支付成功”事件发生时，可能触发“处理支付”动作，并将订单状态从“待支付”转变为“待发货”。

如果业务流程包含超过三个状态（例如订单状态、商品状态），就可以考虑是否使用状态机了。

状态机与流程引擎的主要区别：

1. 状态机关注于对象的状态转换；流程引擎更多地是关注整个业务流程的组织和执行。
2. 状态机结构相对简单，特别适合清晰、固定的流程；流程引擎支持多层次、多路径的复杂流程，允许动态调整和扩展。
3. 状态机自动根据输入或事件改变状态，不需要人工干预；流程引擎则常涉及人的参与决策和协同工作。

在复杂系统中，流程引擎负责高层次的流程管理，而状态机处理具体对象的状态转换，两者互补，共同提高系统的效率和可维护性。

对于 Spring 项目来说，Spring StateMachine 就是开箱即用的状态机实现，功能非常强大。不过，Spring StateMachine 上手难度比较大，还会增加项目的复杂性（有点笨重）。因此，对于状态机功能性需求要求不高的场景（绝大部分项目要求都不高的），不建议使用。并且，由于 Spring StateMachine 状态机实例不是无状态的（包含了上下文），无法做到线程安全，所以代码要么需要使用锁同步，要么需要用 `Threadlocal`，非常的痛苦和难用。 每一次状态机在接收请求的时候，都不得不重新 Build 一个新的状态机实例，导致性能也会比较差，浪费资源。

这里推荐另外一款阿里大佬开源的状态机 Cola-StateMachine，这是一种轻量级的、无状态的、基于注解的状态机实现。由于无状态的特点，可以做到线程安全。

Cola-StateMachine 使用 Java 实现，最简单，实现成本也最低，但是不支持“外部配置”。

Cola-StateMachine 状态流转代码示例：

```
builder.externalTransition()
  .from(States.STATE1)
  .to(States.STATE2)
  .on(Events.EVENT1)
  .when(checkCondition())
  .perform(doAction());
```

这个项目的作者专门写了一篇文章来介绍这个项目：https://blog.csdn.net/significantfrank/article/details/104996419 。

JDK和常见框架中设计模式应用

Spring 使用了哪些设计模式?

- **工厂模式 & 单例模式**：Spring 使用工厂模式可以通过 `BeanFactory` 或 `ApplicationContext` 创建 bean 对象。Spring 中 bean 的默认作用域是 singleton(单例)的。
- **代理模式**：Spring AOP 就是基于动态代理的，如果要代理的对象，实现了某个接口，那么 Spring AOP 会使用 JDK Proxy，去创建代理对象，而对于没有实现接口的对象，就无法使用 JDK Proxy 去进行代理了，这时候 Spring AOP 会使用 Cglib 生成一个被代理对象的子类来作为代理，
- **模板方法模式**：Spring 中 `JdbcTemplate` 、 `HibernateTemplate` 等以 Template 结尾的对数据库操作的类，它们就使用到了模板模式。一般情况下，我们都是使用继承的方式来实现模板模式，但是 Spring 并没有使用这种方式，而是使用 Callback 模式与模板方法模式配合，既达到了代码复用的效果，同时增加了灵活性。
- **装饰者模式**：我们的项目需要连接多个数据库，而且不同的客户在每次访问中根据需要会去访问不同的数据库。这种模式让我们可以根据客户的需求能够动态切换不同的数据源。
- **观察者模式:** Spring的事件驱动模型（ `ApplicationEvent` 和 `ApplicationListener` ）是观察者模式的经典实现。
- **适配器模式** :Spring AOP 的增强或通知(Advice)使用到了适配器模式、Spring MVC 中也是用到了适配器模式适配 `Controller` 。
- ……

详细介绍可以参考笔者写的这篇：Spring 中的设计模式详解。

JDK 使用了哪些设计模式?

IO 流中用到了下面这些设计模式：

- **装饰器模式**：装饰器模式通过组合替代继承来扩展原始类的功能，在一些继承关系比较复杂的场景（IO 这一场景各种类的继承关系就比较复杂）更加实用。对于字节流来说，`FilterInputStream` （对应输入流）和 `FilterOutputStream` （对应输出流）是装饰器模式的核心，分别用于增强 `InputStream` 和 `OutputStream` 子类对象的功能。我们常见的 `BufferedInputStream` (字节缓冲输入流)、 `DataInputStream` 等等都是 `FilterInputStream` 的子类，`BufferedOutputStream` （字节缓冲输出流）、 `DataOutputStream` 等等都是 `FilterOutputStream` 的子类。
- **适配器模式**：适配器模式主要用于接口互不兼容的类的协调工作，你可以将其联想到我们日常经常使用的电源适配器。 `InputStreamReader` 和 `OutputStreamWriter` 就是两个适配器(Adapter)，同时，它们两个也是字节流和字符流之间的桥梁。 `InputStreamReader` 使用 `StreamDecoder` （流解码器）对字节进行解码，**实现字节流到字符流的转换，** `OutputStreamWriter` 使用

`StreamEncoder`（流编码器）对字符进行编码，实现字符流到字节流的转换。

- **工厂模式**：工厂模式用于创建对象，NIO 中大量用到了工厂模式，比如 `Files` 类的 `newInputStream` 方法用于创建 `InputStream` 对象（静态工厂）、 `Paths` 类的 `get` 方法创建 `Path` 对象（静态工厂）、 `ZipFileSystem` 类（ `sun.nio` 包下的类，属于 `java.nio` 相关的一些内部实现）的 `getPath` 的方法创建 `Path` 对象（简单工厂）。
- **观察者模式**：NIO 中的文件目录监听服务基于 `WatchService` 接口和 `Watchable` 接口。 `WatchService` 属于观察者， `Watchable` 属于被观察者。

关于 IO 流中用到的这些设计模式的详细介绍，可以参考笔者写的这篇：Java IO 设计模式总结。

除了 IO 中的这些设计模式之外，还有下面这些设计模式被运用：

- **建造者模式**：JDK中的 `StringBuilder` 和 `StringBuffer` 用到了建造者模式。通过一系列的 `append()` 方法（构建过程）来添加部件，最后调用 `toString()` （构建）得到最终产品。这种方式让复杂对象的构建过程清晰可控，可读性极高。
- **策略模式**： `java.util.Comparator` 接口用到了策略模式， `Collections.sort()` 或 `Arrays.sort()` 方法可以接收一个 `Comparator` 作为参数。这个 `Comparator` 就是策略接口，我们可以传入不同的实现（具体策略）来决定集合到底按什么规则排序。它将“排序”这个行为和“具体排序算法”完全分离开来，非常灵活。
- **桥接模式**： `java.sql` 包提供了一套标准的API接口（如 `Connection` ， `Statement` ），这是抽象部分。而各大数据库厂商（MySQL, Oracle等）根据这套接口提供的具体驱动jar包，则是实现部分。JDBC通过一个 `DriverManager` 将这两部分“桥接”起来。这样，我们的应用程序代码只依赖于标准的抽象接口，无需关心底层用的是哪个数据库的实现，实现了抽象与实现的完美解耦。
- **命令模式**： `Runnable` 接口封装了一个任务（命令）， `Thread` 作为执行者调用 `run()` 方法，实现任务与执行线程的解耦。
- **迭代器模式**：所有集合类（ `List` 、 `Set` 等）都实现 `iterator()` 方法，返回 `Iterator` 实例，客户端通过 `hasNext()` / `next()` 遍历元素，与集合的具体实现（如 `ArrayList` 、 `LinkedList` ）无关。

Netty 使用了哪些设计模式?

Netty 作为高性能的网络通信框架，在源码中大量运用设计模式以保证代码的灵活性、可扩展性和可维护性：

- **单例模式**： `DefaultSelectStrategy` （选择策略）通过 `static final INSTANCE` 直接初始化，类加载时创建唯一实例，供所有 `NioEventLoop` 共享，避免了重复创建。

- **工厂方法模式**：`ReflectiveChannelFactory` 作为 `Channel` 的工厂类，通过反射根据传入的 `Class` 参数动态创建不同类型的 `Channel` （如服务端 `NioServerSocketChannel` 、客户端 `NioSocketChannel` ），避免为每种 `Channel` 单独创建工厂类。
- **责任链模式**：Netty 将网络事件（如连接建立、数据读写）的处理流程，抽象成了一条处理链（ `Pipeline` ）。这条链由一个个的处理器（ `ChannelHandler` ）组成，每个处理器负责一项特定的任务，比如解码、编码、业务逻辑处理等。当一个事件进来时，它会沿着这条链，从头到尾依次被处理器处理。
- **观察者模式**：Netty的所有I/O操作都是异步的。那么，我们如何知道一个异步操作（比如发送数据）何时完成、是成功还是失败呢？Netty 通过 `ChannelFuture` 解决了这个问题。 `ChannelFuture` 作为被观察者，通过 `addListener()` 注册观察者（ `GenericFutureListener` ），当异步操作（如 `writeAndFlush` ）完成时，自动触发所有监听器的 `operationComplete()` 方法，回调处理结果。
- **建造者模式**： `ServerBootstrap` 与 `Bootstrap` 作为客户端和服务端的引导器，通过链式方法配置核心参数（如 `group()` 设置线程池、 `channel()` 指定 Channel 类型、 `childHandler()` 注册处理器），最终通过 `bind()` 或 `connect()` 启动，清晰且灵活。
- **策略模式**： `EventExecutorChooser` 根据 `EventLoopGroup` 中线程池的大小（是否为 2 的幂），动态选择 `PowerOfTwoEventExecutorChooser` （高效取模）或 `GenericEventExecutorChooser` （普通取模），优化 `EventLoop` 的选择性能。
- **装饰者模式**： `WrappedByteBuf` 作为所有 ByteBuf 装饰器的基类，包装原始 `ByteBuf` 实例。子类如 `UnreleasableByteBuf` 增强 `release()` 方法（返回 `false` 禁止释放），`SimpleLeakAwareByteBuf` 增加内存泄漏检测，在不修改原 `ByteBuf` 的前提下扩展功能。

Dubbo 中用到了哪些设计模式?

Apache Dubbo 作为成熟的微服务框架，其源码通过大量设计模式解决了分布式场景下的灵活性、可扩展性和耦合性问题：

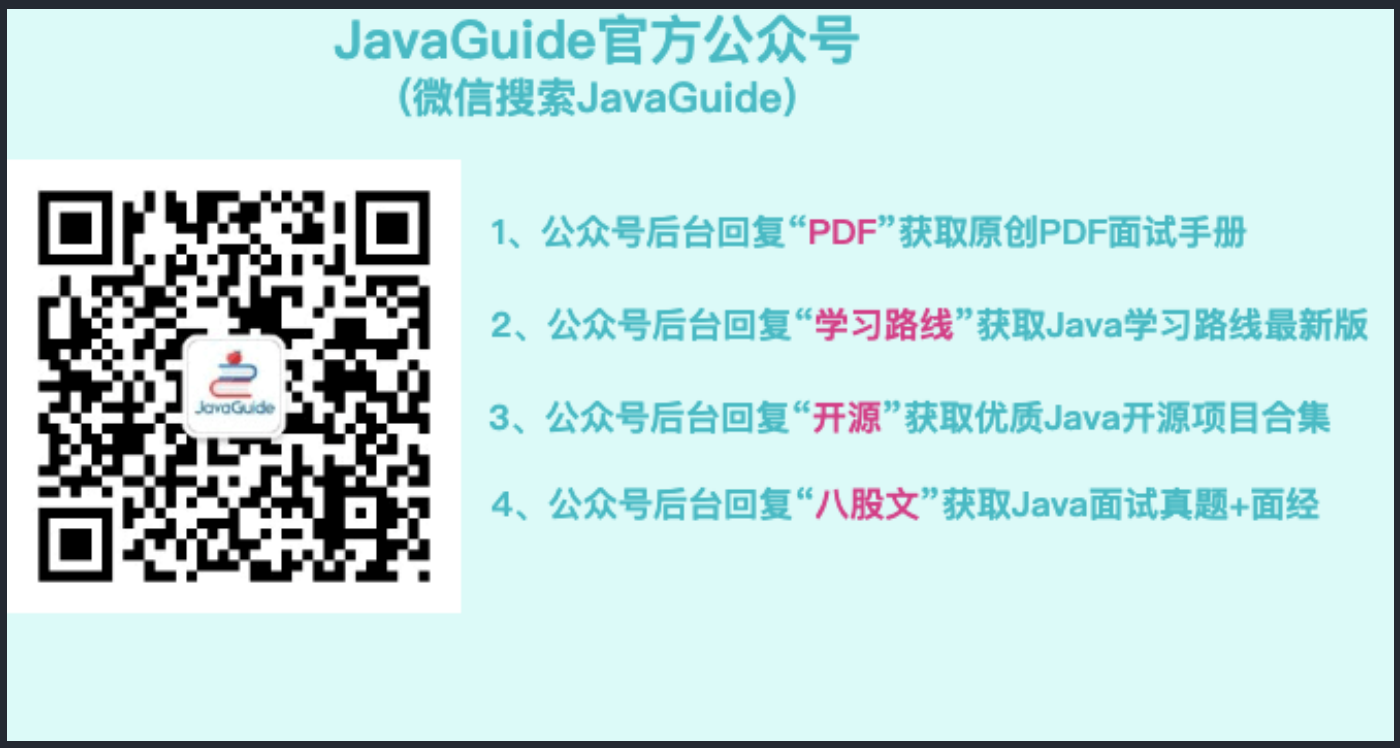
- **单例模式**：作为 Dubbo SPI 机制的核心， `ExtensionLoader` 通过 `ConcurrentMap` 缓存每种类型的加载器实例，确保对同一扩展点类型仅存在一个加载器，避免重复创建导致的资源浪费和状态不一致。
- **工厂模式**： `ExtensionLoader#createExtension` 根据名字实例化具体扩展类属于简单工厂，`ProxyFactory#getProxy / getInvoker` ，由 `JavassistProxyFactory` 、 `StubProxyFactoryWrapper` 等子类决定创建逻辑属于工厂方法。

- **代理模式**：当我们在消费端引用一个远程服务时，我们得到的其实不是一个真实的远程对象，而是一个本地代理对象。`ProxyFactory`（通常是`JavassistProxyFactory`）负责创建这个代理。我们对这个代理对象的方法调用，会被代理逻辑拦截，然后封装成 RPC 请求发送出去。这使得远程调用对用户来说就像本地调用一样透明，是实现RPC的基础。
- **模板方法模式**：为了规范各种协议（`Protocol`）和注册中心（`Registry`）的实现，Dubbo提供了大量的抽象基类。这些基类使用了模板方法模式，定义了核心流程的**骨架**（比如`export`、`refer`、`register`），并把其中易变的部分（如`doExport`、`doRefer`）声明为抽象方法，交由子类去实现。
- **装饰器模式**：`MockClusterInvoker`（Mock 装饰器）包装原始`Invoker`，在服务调用失败时自动触发 mock 逻辑（如返回预设结果），增强了容错能力。装饰器与原始`Invoker`实现同一接口，客户端无需感知差异。
- **策略模式**：`LoadBalance`接口定义服务选择算法，具体实现（`RandomLoadBalance`随机、`RoundRobinLoadBalance`轮询等）封装不同负载均衡逻辑。框架根据配置（如`loadbalance=random`）动态选择策略，支持灵活切换。
- **责任链模式**：`ProtocolFilterWrapper`通过`buildInvokerChain()`构建过滤器链，将多个`Filter`（如监控、日志、限流、认证等）按顺序封装为`Invoker`链。请求经过每个过滤器时，可修改参数或中断传递，最终到达目标服务。

面试资料

整理汇总了一些 Java 面试相关的高质量 PDF 资料，涵盖 Java 基础、并发、JVM、设计模式、数据库、SpringBoot、分布式、消息队列、智力题……。

如何获取？ 在我的公众号“**JavaGuide**”后台回复“**PDF**” 即可获取！



20:07

面试

Java 面试资料汇总链接: https://pan.baidu.com/s/1oIABHaZRCBGtWpSi3MnzcA 提取码: s343